Lahore University of Management Sciences
AI651 – Deep Learning for Space, Time and Graphs, Fall 2026

Assignment 2

Muhammad Adnan
25280067

Code repository (public):
`https://github.com/adnan821/DL4STG-PA2`

Hardware for all experiments: NVIDIA GeForce RTX 3060 (12 GB), PyTorch 2.7.1 + CUDA 11.8

Question 1 — River Forecasting Proposal

I propose a small spatiotemporal graph network with seven river nodes. Every 10 minutes, it predicts water levels at S1–S4 one hour ahead and reports an uncertainty band. The model is designed to run on a CPU.

1. Architecture diagram

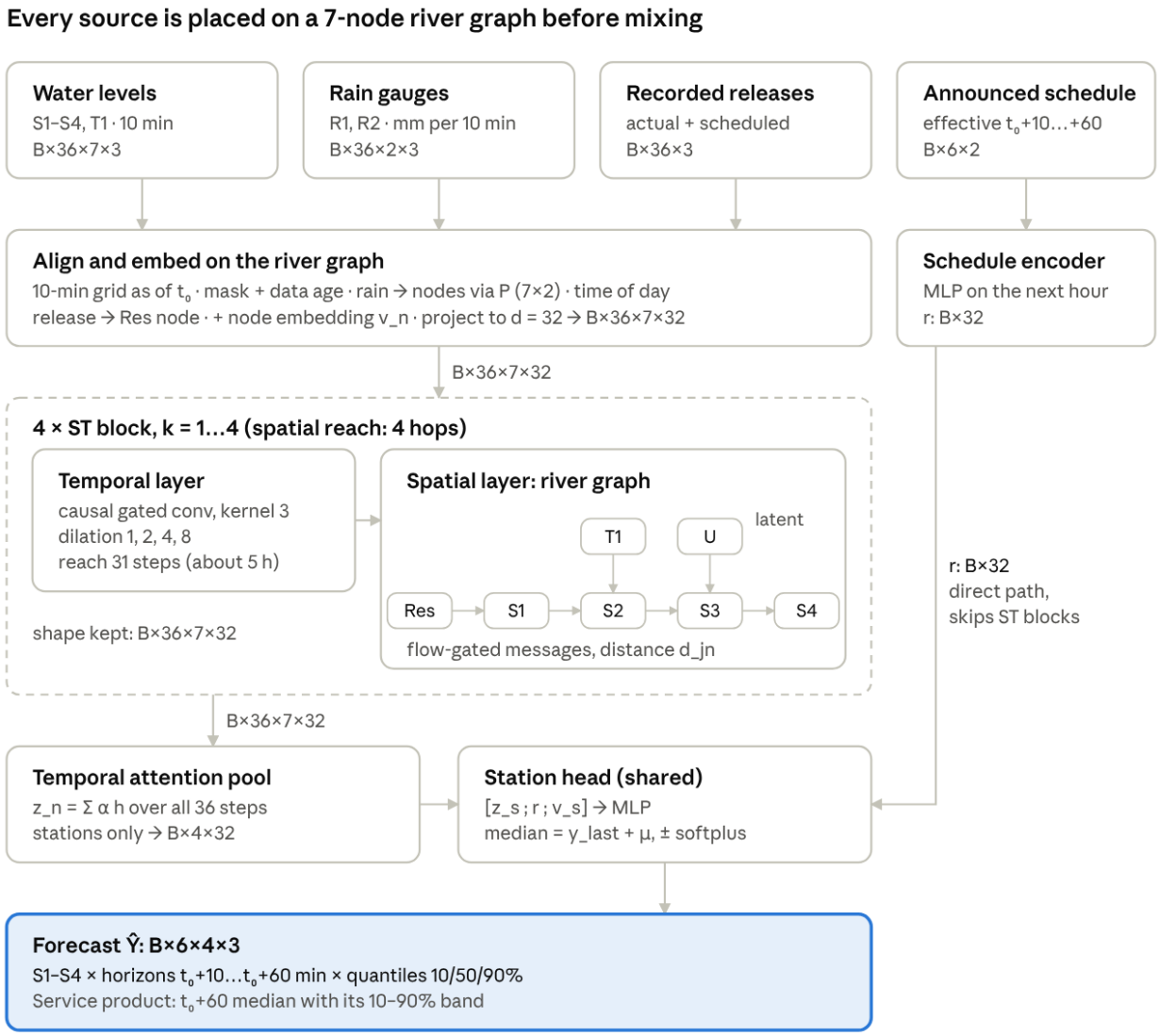


Figure Q1: Forecasting architecture. B = batch of issuance times. Sources are aligned to one 10 minute grid as of t_0 and placed on graph nodes; four temporal + spatial blocks carry upstream information down the river; the announced release schedule bypasses the blocks and enters every station head directly. U is a latent node for the unmonitored tributary.

2. Assumptions

Supplied facts.

- Levels at S1–S4 and T1 are recorded every 10 minutes. Rainfall at R1 and R2 includes timestamps and gauge locations.
- Reservoir records include actual releases and announced schedules, with announcement and effective times.
- Locations, river connections and distances are known. Main river flow is Res $\rightarrow$ S1 $\rightarrow$ S2 $\rightarrow$ S3 $\rightarrow$ S4.
- T1 joins between S1 and S2. The unmonitored tributary joins between S2 and S3.
- Travel times vary and are not supplied. Future rain and water levels are unavailable when a forecast is issued.

ID	Assumption	Why reasonable	Effect on design
A1	At least 3 years of 10 minute history exist, including several flood events.	Authorities typically archive telemetry for years.	Enough data for a small learned model; train/validation/test split by year so floods do not leak across splits.
A2	Travel time from the reservoir to S4 is at most about 4 h, even at low flow.	A regulated river of a few tens of km at 1–3 m/s; to be checked against the supplied distances.	History window $L = 36$ steps (6 h) covers the slowest travel time plus a margin.
A3	Rainfall reaches the river within 0–6 h, faster for the unmonitored catchment near S2–S3.	Typical response of small to medium catchments.	Rain history uses the same 6 h window; no future rain is needed for a 1 h horizon.
A4	Water moves only downstream; backwater effects at confluences are small.	Typical for non tidal rivers with a clear slope.	Directed graph edges point downstream only, plus self loops.
A5	Travel time shrinks as flow rises.	Standard hydraulics: wave speed increases with depth.	Edge messages are gated by the node states, so learned delays can change with flow.
A6	Telemetry arrives with 0–20 min delay; about 1–5% of readings are missing, with occasional outages of hours.	Common for radio/cellular gauges.	Masks and data age features on every input; training with simulated gaps and delays.
A7	Level sensors are accurate to about 1–2 cm; occasional spikes and flat readings occur.	Typical pressure or radar gauges.	Range, rate of change and flat reading checks set bad readings to missing before the model.
A8	Announced schedules are mostly followed, but actual releases can deviate in timing or volume.	Operators adjust to demand and safety.	Separate inputs for scheduled and actual releases, and their recent difference.
A9	R1 and R2 give a usable but incomplete picture of catchment rainfall, including the unmonitored tributary.	Two point gauges cannot capture convective storms.	Rain mapped to nodes with fixed area based weights; the unmonitored inflow is a latent node driven by rain.
A10	Rating curves and station datums are stable over the record.	Usual unless the channel changes after floods.	Levels used directly, normalised per station; drift listed as a limitation.
A11	All four stations are forecast at once, every 10 min, on modest hardware.	Stated service.	One shared model for all stations, small enough for a CPU.

3. Forecasting task and data specification

At each issuance time t_0 (every 10 minutes), the model uses everything that has *arrived* by t_0 over the last 6 h and outputs levels at S1–S4 for $t_0 + 10, \ldots, t_0 + 60$ min as three quantiles each. The service forecast is the $t_0 + 60$ median with its 10–90% band.

- **Issuance time** t_0: on the 10 minute grid. Only data with arrival time $\leq t_0$ is used (an "as of" rule).
- **History window**: $L = 36$ ten minute bins ending at t_0, stamped $t_0 - 350$ min, $\ldots, t_0$; each bin covers the 10 minutes up to its stamp, so together they cover the 6 h interval $(t_0 - 360\,\text{min}, t_0]$ (A2, A3).
- **Horizon**: $H = 6$ steps, $t_0+10, \ldots, t_0+60$ min. The 10–50 min outputs are auxiliary training targets; the 60 min output is the product.
- **Graph**: $N = 7$ nodes in fixed order {Res, S1, T1, S2, U, S3, S4}. U is a latent node at the unmonitored confluence: it has no level data, only rain and a learned embedding.

Tensor	Shape	Axes	Time coverage
Levels X^{lev}	$B \times L \times N \times 3$	batch, time step, node, [normalised level, observed mask, data age in steps]	$t_0 - 35, \ldots, t_0$; Res and U rows always masked
Rain X^{rain}	$B \times L \times 2 \times 3$	batch, step, gauge (R1, R2), [rain mm/10 min, mask, age]	$t_0 - 35, \ldots, t_0$
Recorded releases X^{rel}	$B \times L \times 3$	batch, step, [actual release m^3/s, mask, scheduled release in force]	$t_0 - 35, \ldots, t_0$
Announced schedule X^{sch}	$B \times H \times 2$	batch, future step, [scheduled release, announced by t_0 flag]	$t_0 + 1, \ldots, t_0 + 6$
Calendar X^{cal}	$B \times L \times 2$	batch, step, [sin, cos of time of day]	$t_0 - 35, \ldots, t_0$
Static graph	$A \in \{0,1\}^{N \times N}$, $D \in \mathbb{R}^{N \times N}$	directed adjacency (downstream + self loops); along river distance (km)	constant
Rain to node map P	$N \times 2$	node, gauge: share of each node's local catchment rain taken from each gauge	constant
Output $\hat{Y}$	$B \times H \times 4 \times 3$	batch, horizon, station (S1–S4), quantile (10, 50, 90%)	$t_0 + 1, \ldots, t_0 + 6$

Alignment and availability.

- **Time:** align all sources to bins of 10 minutes ending at t_0. Take the level reading in each bin, sum rainfall and average releases.
- **Gaps:** carry forward the last valid value. Set mask = 0 and record its age for missing readings or readings that have not arrived.
- **Location:** map rainfall to river nodes using P and releases to Res. Align schedules by their effective time, using only versions announced by t_0.
- **Available:** levels, rain and actual releases received by t_0, plus schedules announced by then.
- **Unavailable:** future rain, future levels, unannounced releases and the unmonitored tributary's inflow.

4. Model computations and design justification

The model maps inputs to river nodes and applies four temporal and spatial blocks. Four spatial hops connect Res to S4. The forecast head predicts quantiles of the change from the last observed level.

Notation. n = node, τ = history step $(1, \dots, L)$, h = horizon $(1, \dots, 6)$, $q \in \{0.1, 0.5, 0.9\}$, $d = 32$ hidden channels, $K = 4$ blocks, $\mathcal{N}^-(n)$ = upstream neighbours of n plus n itself, d_{jn} = along river distance from j to n.

(a) Input projection (Fig. Q1, "Align and embed"). Each source is projected to d channels at its node; rain is mapped to nodes with P, releases enter only at Res, and v_n is a learned node embedding (for U it stands in for the unobserved inflow):

$$e_{n,\tau} = W_{lev}\, x^{lev}_{n,\tau} + W_{rain}\,(P\, x^{rain}_{\tau})_n + \mathbb{1}[n = \text{Res}]\, W_{rel}\, x^{rel}_{\tau} + W_{cal}\, x^{cal}_{\tau} + v_n. \tag{1}$$

(b) Temporal layer in block k: a causal gated convolution along τ, kernel 3, dilation 2^{k-1} (1, 2, 4, 8), so four blocks see $1 + 2(1 + 2 + 4 + 8) = 31$ steps:

$$\tilde{h}_{n,\cdot} = \big(W_a * h_{n,\cdot}\big) \odot \sigma\big(W_b * h_{n,\cdot}\big). \tag{2}$$

(c) Spatial layer in block k: directed message passing along river edges. The gate depends on both nodes' states and the distance, so the effective delay and attenuation can change with flow (A5):

$$m_{n,\tau} = \sum_{j \in \mathcal{N}^-(n)} \sigma\big(g([\tilde{h}_{j,\tau}; \tilde{h}_{n,\tau}; d_{jn}])\big) \odot W_s\, \tilde{h}_{j,\tau}, \tag{3}$$

$$h_{n,\tau} \leftarrow \text{LN}\big(\tilde{h}_{n,\tau} + \text{ReLU}(m_{n,\tau})\big). \tag{4}$$

(d) Temporal readout. Attention pooling over all $L = 36$ steps of the last block covers the steps the convolutions do not reach:

$$z_n = \sum_{\tau=1}^{L} \alpha_{n,\tau}\, h_{n,\tau}, \qquad \alpha_{n,\cdot} = \text{softmax}_\tau\big(w^\top h_{n,\tau}\big). \tag{5}$$

(e) Release branch and forecast head. The announced schedule is embedded once as $r = \text{MLP}(X^{sch})$, a vector in $\mathbb{R}^d$. It is fed *directly* to every station head without passing through the four blocks. For station s:

$$[\mu_{s,h},\, a_{s,h},\, b_{s,h}] = \text{MLP}_{head}([z_s;\, r;\, v_s]), \tag{6}$$

$$\hat{y}^{0.5}_{s,h} = y^{last}_s + \mu_{s,h}, \tag{7}$$

$$\hat{y}^{0.1}_{s,h} = \hat{y}^{0.5}_{s,h} - \text{softplus}(a_{s,h}), \quad \hat{y}^{0.9}_{s,h} = \hat{y}^{0.5}_{s,h} + \text{softplus}(b_{s,h}). \tag{8}$$

Predicting the change from the last reading makes persistence the starting point; softplus keeps the quantiles ordered.

(f) Training objective. Weighted, masked pinball loss over stations, horizons and quantiles, with $\rho_q(e) = \max\big(q\,e,\ (q-1)\,e\big)$:

$$\mathcal{L} = \frac{1}{\sum M} \sum_{b,s,h,q} M_{b,s,h}\, w_h\, w_{b,s,h}\, \rho_q\big(y_{b,s,h} - \hat{y}^q_{b,s,h}\big), \tag{9}$$

$$w_h = 1 + \mathbb{1}[h = 6], \qquad w_{b,s,h} = 1 + \lambda \mathbb{1}[y_{b,s,h} > y_s^{alert}]. \tag{10}$$

M masks missing targets; the flood weight λ (e.g. 3) keeps rare high water from being averaged away. Training uses Adam on years not used for validation or test, and five seeds are kept as an ensemble.

Why these choices.

- **Directed graph:** river connections define neighbours; station indices do not. This handles uneven spacing and downstream flow (A4).
- **Four spatial layers:** the longest path, Res to S4, has four hops.
- **Six hours of history:** covers the assumed travel and rainfall response times (A2, A3).
- **Small model:** width $d = 32$ limits computation and overfitting on a graph with only seven nodes.
- **Quantiles:** report a range of plausible levels because the unmonitored tributary and future rain leave part of the outcome unknown.

How each source reaches a forecast.

- **Upstream conditions**: S1's state reaches S2, S3, S4 through 1, 2, 3 spatial hops, delayed by the temporal layers.
- **Monitored tributary**: T1 is its own node with an edge into S2.
- **Unmonitored tributary**: node U (rain via P plus its embedding) with an edge into S3; its effect is learned only indirectly, from S3 levels that S2 cannot explain.
- **Rainfall**: enters at every node in proportion to P, and drives U.
- **Releases**: past releases propagate from Res down the graph; announced future releases reach every head directly through r.

Physical assumptions vs learned mappings.

Imposed by design (physical)	Learned from data
Water flows downstream only (directed edges)	Travel times and how they change with flow
Effects are causal in time (causal convolutions, as of rule)	Attenuation and mixing at each confluence
Only river connected stations exchange information (graph edges)	How influence varies with distance (distance is an input, not a constraint)
	Rainfall–runoff response per catchment
Rain falls where the gauges map it (P)	Unmonitored tributary inflow (via U)
Persistence as the default forecast (residual)	Station specific rating behaviour, seasonality

5. Operation and exceptional conditions

The service continues issuing forecasts during gaps and delays, with a degraded status when inputs are stale. Training encourages wider uncertainty bands under missing data; validation must check that behaviour.

(i) Missing observations.

- Treat failed quality checks and missing readings alike: carry forward the last valid value, set mask $= 0$ and record its age.
- Simulate isolated gaps and outages lasting hours during training. Check whether the model uses neighbouring stations and widens its intervals appropriately.
- If a station has no readings within the six hour window, issue a forecast using the remaining inputs and flag it as degraded. Exclude missing targets from the loss.

(ii) Delayed observations.

- Use only data received by t_0. Treat late readings as missing until they arrive.
- Include late data in the next forecast cycle. Simulate station delays during training using arrival logs where available (A6).

(iii) Announced future releases.

- Select the latest schedule announced by t_0. Map releases to X^{sch} by effective time and flag which steps are covered.
- Use historical schedule versions during training, so the model sees the information that was available at each issuance time.
- Feed the schedule directly to each forecast head through r. A release scheduled for $t_0 + 20$ can therefore affect the forecast issued now.

(iv) Scheduled vs actual releases.

- Include both actual and scheduled past releases in X^{rel} so the model can use their discrepancy.
- Update the forecast in the next cycle once actual releases arrive. Flag deviations above an operational threshold, such as 10%, for the operator.

Feasibility of running every 10 minutes.

- Each run processes seven nodes, 36 steps and 32 channels through four blocks. The estimated size is about 40k parameters and a few million multiply and add operations.
- This should fit comfortably within the 10 minute cycle on a CPU, including an ensemble of five models. These are estimates; no implementation or runtime benchmark is claimed.
- Retrain offline, for example each season, and validate on recent events before replacing the deployed model.

6. Uncertainty, limitations, and additional information

The unmonitored tributary is the main unresolved source of uncertainty for S3 and S4.

How uncertainty is represented.

- Predict the 10th, 50th and 90th percentiles to describe uncertainty in future levels.
- Use variation across five training seeds as an indication of model uncertainty.
- Flood weighting changes the quantiles learned by the loss. Recalibrate the bands on validation years, for example with conformal rescaling, and check coverage against the 80% target.

Uncertainty or limitation	Effect on the forecast	Response, or what stays unresolved
Unmonitored tributary between S2 and S3	S3 and S4 errors larger; peaks underestimated when it floods	Latent node U driven by R1/R2 and wider quantiles; unresolved without a gauge
Travel times change with flow	Peak timing off, worst in rare high flows	Flow gated edges and 6 h history; few large floods to learn from remains a limit
Only two rain gauges	Local storms missed, so rises start late	Check interval coverage during storms; radar could improve rainfall coverage
No future rain	Rain during the next hour is unknown	Little effect at 1 h for larger catchments; small fast catchments near S3 stay uncertain
Floods beyond the training range	Model may underpredict record events	Flood weighted loss and persistence residual; flag forecasts above the highest training level
Releases deviating from schedule	Forecasts issued before the change are wrong	Update next cycle once logged; alert on large deviation
Sensor drift, datum or rating changes	Systematic bias at one station	Quality checks and periodic retraining; unresolved between calibrations
Data delays and outages	Less information, wider bands	Masks, ages and training with simulated gaps

Additional information requests. The system stays usable without every item; each one improves it.

- **Gauge on the unmonitored tributary:** turns U into an observed node; largest gain for S3 and S4 (without it: wider S3/S4 bands).
- **Rainfall radar or short term rain forecasts:** better spatial rain and rain during the next hour.
- **Rating curves and cross sections:** convert levels to discharge, enabling a mass balance check.
- **Travel time or tracer studies:** prior for edge delays; check of learned timing.
- **Reservoir operating rules and schedule change history:** better model of deviations.
- **Telemetry arrival logs and maintenance records:** realistic delay and gap simulation (without them: assumption A6).
- **Catchment boundaries (DEM):** better rain to node weights P (without them: distance based weights).

Question 2 — Motion or Memory? Results

All numbers below use the 1,000 test videos (9,000–9,999), except Table 1 (synthetic tensors), Figure 1 (training videos), parameter counts and timings. Errors are given to three significant figures, with scientific notation below 10^{-3}. Learned models are reported as mean (seed 0, seed 1).

1. Setup

Table 1: Checks of the transformation and window functions.

	Check	Expected	Yours
(a)	$\max \lvert M(M(x)) - x \rvert$	0	0.0 (`torch.equal` = True)
(b)	$\max \lvert R^4(x) - x \rvert$	0	0.0 (`torch.equal` = True)
(c)	$\max \lvert MRM(x) - R^{-1}(x) \rvert$	0	0.0 (`torch.equal` = True)
(d)	bright pixel of $M(p)$ (frame, row, column)	(7, 10, 43)	(7, 10, 43)
(e)	bright pixel of $R(p)$ (frame, row, column)	(7, 43, 10)	(7, 43, 10)
(f)	1× input / target frames	1–5 / 6–10	1, 2, 3, 4, 5 / 6, 7, 8, 9, 10
(g)	2× input / target frames	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19
(h)	3× input / target frames	1, 4, 7, 10, 13 / 16, 19	1, 4, 7, 10, 13 / 16, 19

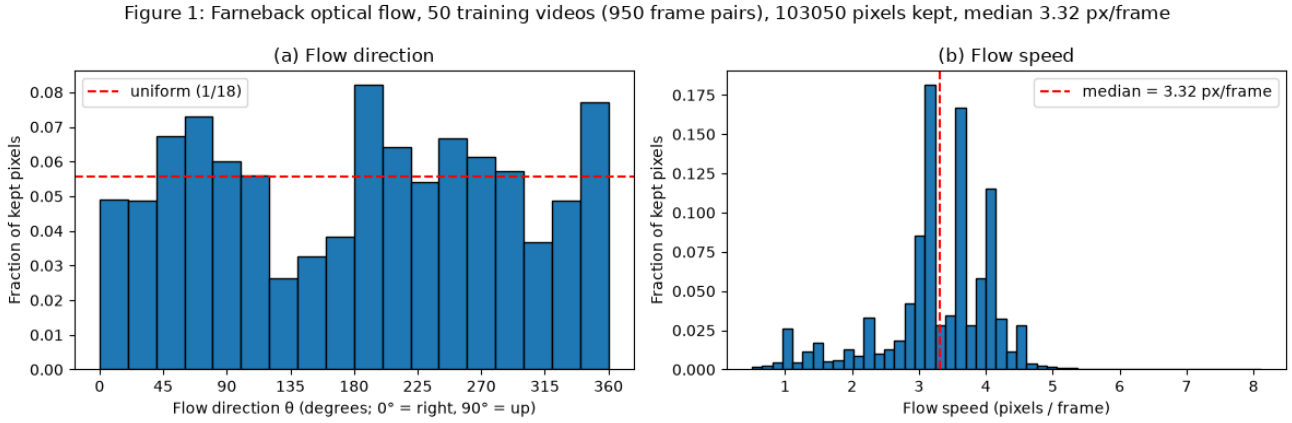


Figure 1: Farneback optical flow on the first 50 training videos (950 frame pairs). (a) Flow direction, fraction of kept pixels per 20° bin; red line = uniform level 1/18. (b) Flow speed; red line = median. **103,050 pixels kept; median speed 3.32 px/frame.**

2. Baseline: ConvLSTM

Table 2: Per horizon test error at $1\times$ ($t = 1$), model A vs reference predictors.

	Model A	Blank	Copy last
MSE_1	0.0166 (0.0170, 0.0162)	0.0426	0.0492
MSE_2	0.0203 (0.0207, 0.0199)	0.0426	0.0604
MSE_3	0.0226 (0.0231, 0.0222)	0.0426	0.0669
MSE_4	0.0244 (0.0249, 0.0240)	0.0425	0.0708
MSE_5	0.0262 (0.0267, 0.0257)	0.0425	0.0732
$E^{(5)}$	**0.0220 (0.0225, 0.0216)**	0.0426	0.0641
Trainable parameters	480,784	0	0

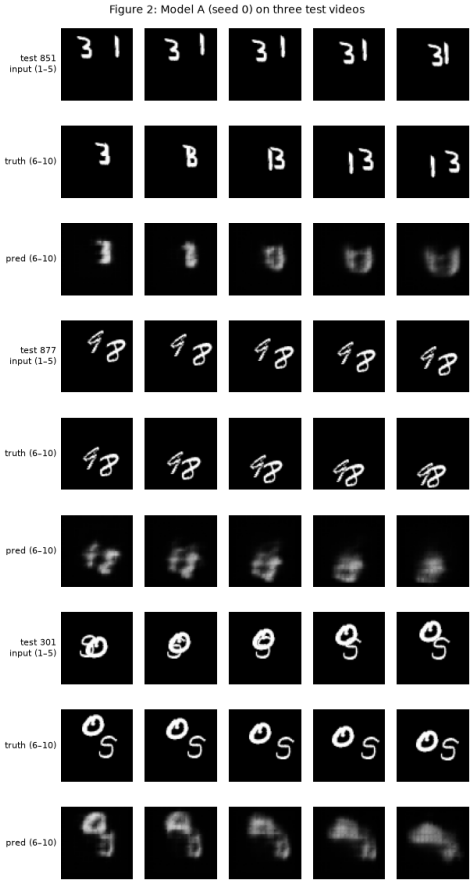


Figure 2: Model A (seed 0) on three test videos: input frames 1–5, ground truth 6–10 and prediction 6–10. Videos 851 and 877 contain overlapping digits; video 301 shows digits that have just separated.

3. Testing the Baseline

Table 3: Consistency gaps and transformed errors at $1\times$, $t = 1$.

	Model A	Blank	Copy last
$E^{(5)}$	0.0220 (0.0225, 0.0216)	0.0426	0.0641
C_M	1.82e-3 (1.77e-3, 1.87e-3)	0	0
E_M	0.0227 (0.0232, 0.0223)	0.0426	0.0641
C_R	2.95e-3 (2.94e-3, 2.96e-3)	0	0
E_R	0.0237 (0.0242, 0.0233)	0.0426	0.0641

Table 4: Model A at faster playback ($t = 1$). Speeds are compared on the first two horizons only.

	Model A	Copy last
$E^{(2)}$ at $1\times$	0.0185 (0.0188, 0.0181)	0.0548
$E^{(2)}$ at $2\times$	0.0457 (0.0448, 0.0466)	0.0661
$E^{(2)}$ at $3\times$	0.0533 (0.0523, 0.0542)	0.0703
$E^{(5)}$ at $1\times$	0.0220 (0.0225, 0.0216)	0.0641
$E^{(5)}$ at $2\times$	0.0501 (0.0493, 0.0510)	0.0719

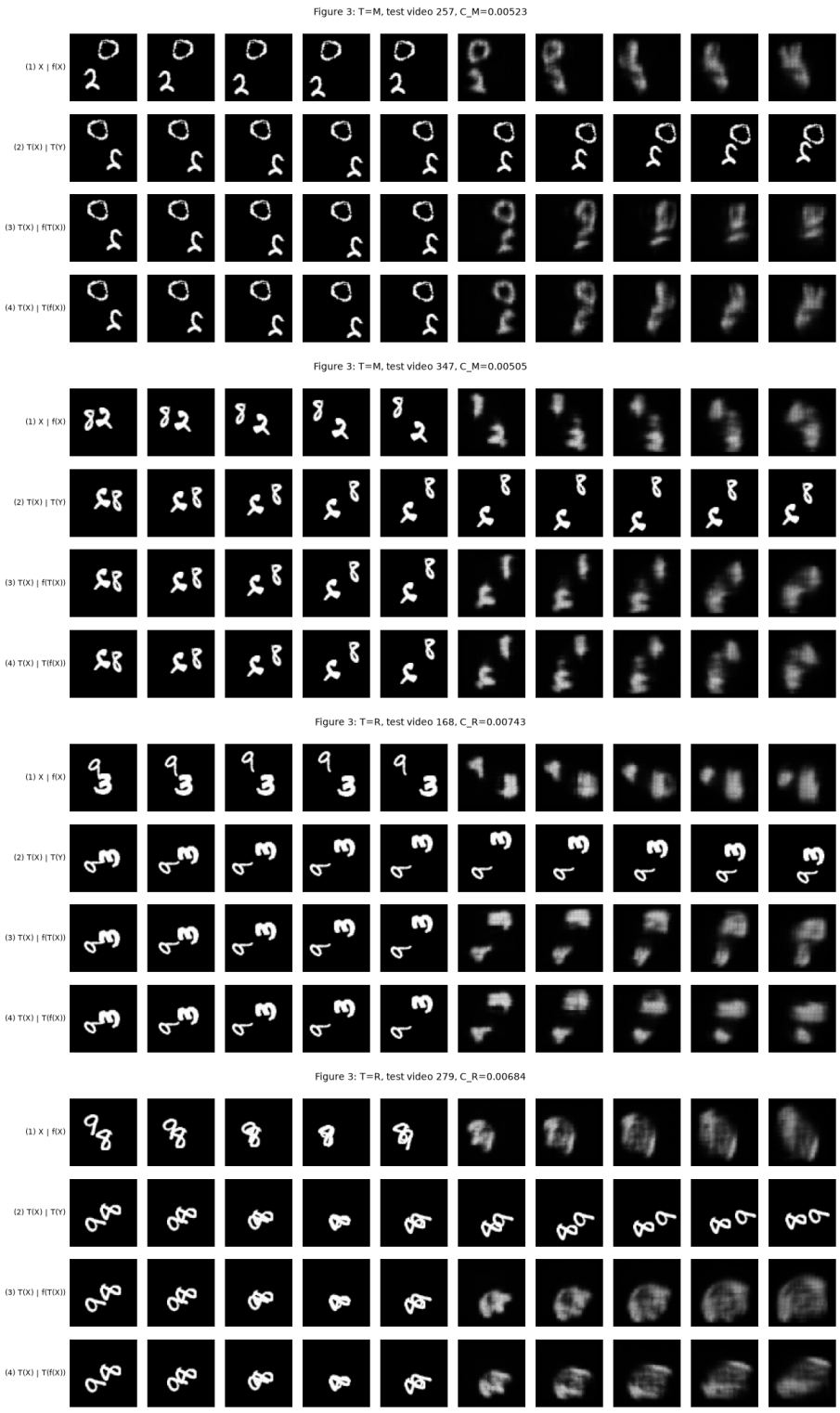


Figure 3: Model A (seed 0). Top two: the test videos with the largest per video C_M ($T = M$), 257 ($C_M = 5.23 \times 10^{-3}$) and 347. Bottom two: the largest per video C_R ($T = R$), 168 ($C_R = 7.43 \times 10^{-3}$) and 279. Rows: (1) $X \,|\, f(X)$; (2) $T(X) \,|\, T(Y)$; (3) $T(X) \,|\, f(T(X))$; (4) $T(X) \,|\, T(f(X))$. GIFs: `gifs/figure3_A_M_test257.gif`, `gifs/figure3_A_M_test347.gif`, `gifs/figure3_A_R_test168.gif`, `gifs/figure3_A_R_test279.gif`.

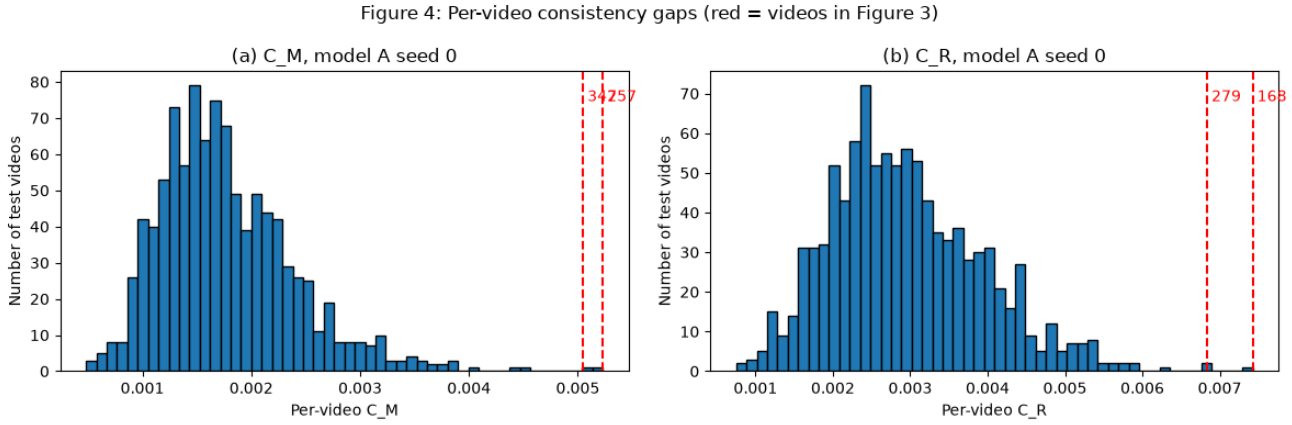


Figure 4: Histograms of per video (a) C_M and (b) C_R for model A (seed 0) over the 1,000 test videos. Red dashed lines mark the videos in Figure 3 (257, 347 for C_M; 168, 279 for C_R).

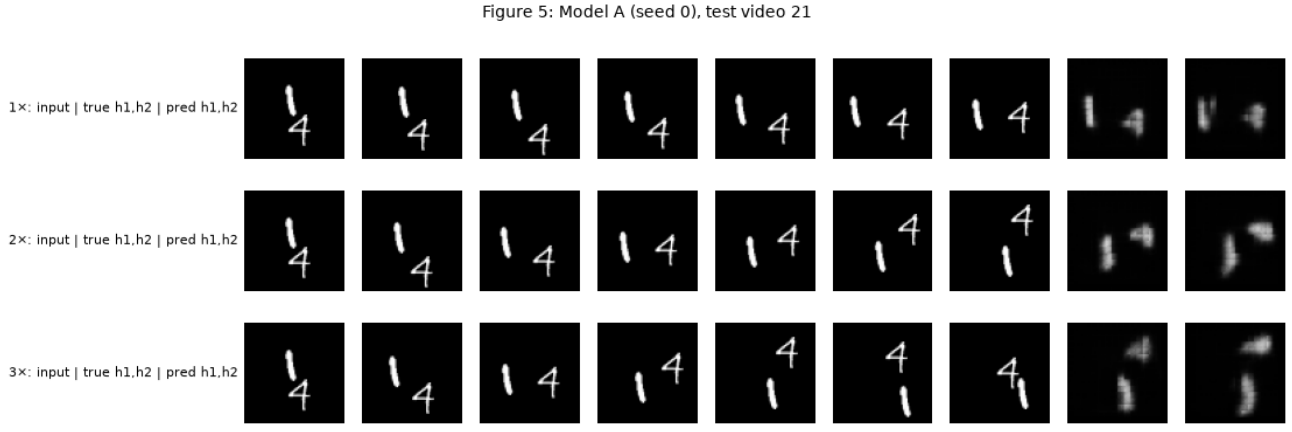


Figure 5: Model A (seed 0) on test video 21 at 1×, 2× and 3×: the five input frames, the first two true future frames, and the first two predicted frames.

4. What Can Data Fix?

Table 5: Models A–D. A: baseline, 1× only. B: + symmetry augmentation. C: + speed augmentation. D: speed augmentation and larger reach (dilations 1, 3). Training time = mean wall clock time per epoch over 30 epochs; inference time = time to predict all 1,000 test videos (1×, $t = 1$) at batch size 32. RTX 3060.

	A	B	C	D
$E^{(5)}$ 1×	0.0220 (0.0225, 0.0216)	0.0227 (0.0234, 0.0219)	0.0266 (0.0269, 0.0263)	0.0300 (0.0301, 0.0300)
C_M	1.82e-3 (1.77e-3, 1.87e-3)	1.08e-3 (9.38e-4, 1.22e-3)	1.03e-3 (9.08e-4, 1.16e-3)	4.44e-4 (3.88e-4, 4.99e-4)
E_M	0.0227 (0.0232, 0.0223)	0.0226 (0.0233, 0.0218)	0.0268 (0.0271, 0.0265)	0.0301 (0.0301, 0.0300)
C_R	2.95e-3 (2.94e-3, 2.96e-3)	1.44e-3 (1.22e-3, 1.66e-3)	2.12e-3 (1.87e-3, 2.37e-3)	8.60e-4 (6.62e-4, 1.06e-3)
E_R	0.0237 (0.0242, 0.0233)	0.0227 (0.0235, 0.0218)	0.0278 (0.0279, 0.0276)	0.0304 (0.0305, 0.0304)
$E^{(2)}$ 1×	0.0185 (0.0188, 0.0181)	0.0188 (0.0194, 0.0182)	0.0230 (0.0230, 0.0231)	0.0276 (0.0279, 0.0274)
$E^{(2)}$ 2×	0.0457 (0.0448, 0.0466)	0.0438 (0.0421, 0.0456)	0.0290 (0.0292, 0.0287)	0.0308 (0.0309, 0.0307)
$E^{(2)}$ 3×	0.0533 (0.0523, 0.0542)	0.0494 (0.0465, 0.0524)	0.0335 (0.0335, 0.0335)	0.0339 (0.0336, 0.0341)
Parameters	480,784	480,784	480,784	480,784
Train (s/epoch)	14.6 (14.6, 14.6)	15.7 (15.8, 15.7)	15.0 (15.0, 14.9)	14.9 (14.8, 15.0)
Inference (s)	0.378 (0.382, 0.375)	0.373 (0.373, 0.374)	0.373 (0.374, 0.373)	0.369 (0.370, 0.369)

5. The Final Model

Table 6: Symmetrised forecasters $\bar{f}(X) = \frac{1}{8}\sum_{g\in G} g^{-1} f(g(X))$. Final model: symmetrised D. Training time is that of the underlying model; inference includes all 8 transformations, forward passes, inverses and the averaging.

	Sym B	**Sym D (final)**
$E^{(5)}$ 1×	0.0221 (0.0229, 0.0212)	0.0299 (0.0300, 0.0298)
C_M	4.17e-17 (3.55e-17, 4.79e-17)	2.56e-17 (2.30e-17, 2.81e-17)
E_M	0.0221 (0.0229, 0.0212)	0.0299 (0.0300, 0.0298)
C_R	4.28e-17 (3.63e-17, 4.92e-17)	2.62e-17 (2.37e-17, 2.88e-17)
E_R	0.0221 (0.0229, 0.0212)	0.0299 (0.0300, 0.0298)
$E^{(2)}$ 1×	0.0184 (0.0190, 0.0177)	0.0276 (0.0279, 0.0274)
$E^{(2)}$ 2×	0.0429 (0.0414, 0.0445)	0.0308 (0.0309, 0.0306)
$E^{(2)}$ 3×	0.0482 (0.0456, 0.0509)	0.0338 (0.0336, 0.0340)
Parameters	480,784	480,784
Train (s/epoch)	15.7 (15.8, 15.7)	14.9 (14.8, 15.0)
Inference (s)	2.97 (2.97, 2.98)	2.94 (2.95, 2.94)

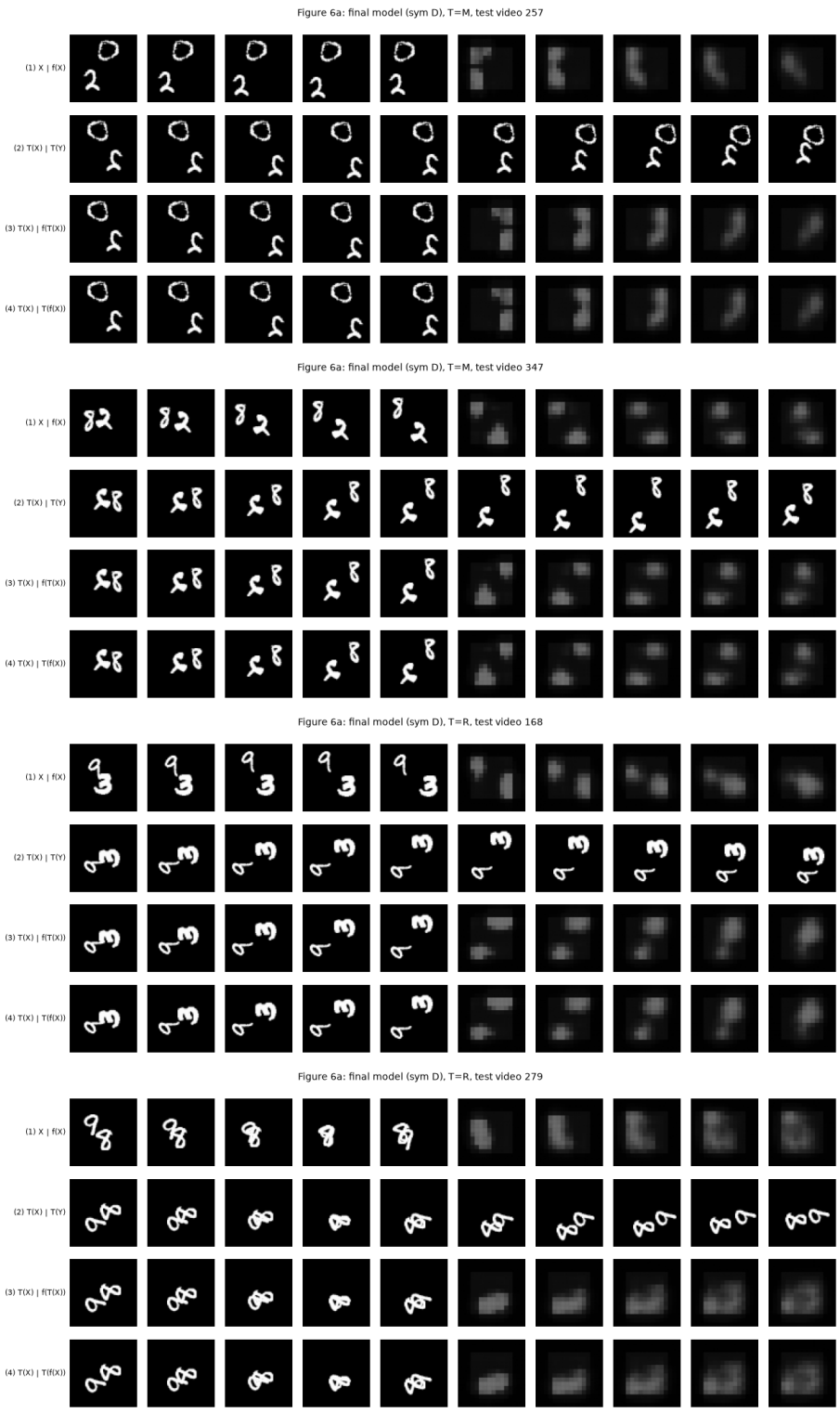


Figure 6: (a) Figure 3 repeated with the final model (symmetrised D, seed 0) on the same videos: $T = M$ for 257 and 347, $T = R$ for 168 and 279. GIFs: `gifs/figure6a_final_M_test257.gif`, `gifs/figure6a_final_M_test347.gif`, `gifs/figure6a_final_R_test168.gif`, `gifs/figure6a_final_R_test279.gif`. (b) on the next figure.

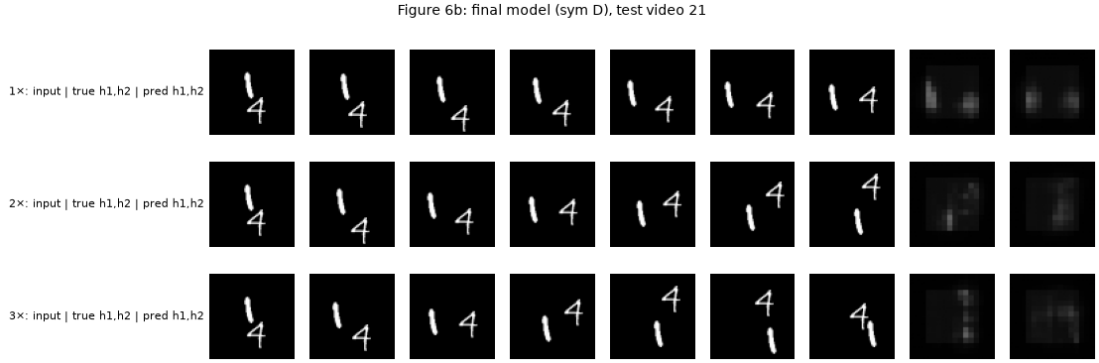


Figure 6: (continued) (b) Figure 5 repeated with the final model on test video 21 at 1×, 2×, 3×.

Question 2 — Written answers

Hardware: NVIDIA GeForce RTX 3060 (12 GB). Values in tables are "mean (seed 0, seed 1)".

1. Setup

Q2.1

- **Direction:** Figure 1a is broadly spread across directions. Bin fractions range from 0.026 to 0.082 around the uniform value $1/18 = 0.056$.
- Mirroring maps θ to $180° - \theta$; rotation maps it to $\theta + 90°$ (modulo 360°). Neither should greatly change this broad distribution, so transformed trajectories should be reasonably familiar.
- **Speed:** the median is 3.32 px/frame across 103,050 retained pixels. Typical speeds at 2× and 3× are therefore about 6.6 and 10 px/frame.
- **Limitations:** the 50 videos contain only about 100 digits, and pooled pixels are not independent samples. Overlap can distort the flow estimates. The plots support approximate symmetry, but do not prove it; checking clips without overlap would test the conclusion.

Q2.2

- **Mirror:** changes digit appearance while largely preserving the observed distribution of motion directions.
- **Rotation:** also preserves valid motion, but changes digit appearance more noticeably.
- **Faster playback:** keeps the digits' shapes while increasing displacement between frames.
- These tests help separate sensitivity to appearance from sensitivity to motion. They suggest causes of failure; the training comparisons in Table 5 provide stronger evidence.

2. Baseline: ConvLSTM

Q2.3 — Reach of model A

- Patch folding: one cell of the 16×16 grid = 4 original pixels.
- Only 3×3 convolutions move information, by 1 cell each. The cell update is elementwise; the 1×1 output convolution and pixel (un)shuffle move nothing.
- Longest path in one step: new input u_τ (or $h^{(1)}_{\tau-1}$) → layer 1 convolution (1 cell) → layer 2 convolution (1 cell) → output. Total 2 cells.
- The path from $h^{(2)}_{\tau-1}$ only passes the layer 2 convolution: 1 cell.
- **Reach = 2 cells × 4 px = 8 px per time step.** An impulse test on the untrained network confirmed 2 cells.

Q2.4 — Model A vs references (Table 2, Fig. 2)

- Model A beats both references at every horizon: $E^{(5)} = 0.0220$ vs 0.0426 (Blank) and 0.0641 (Copy last).
- Error grows steadily with horizon: 0.0166 at $h = 1$ to 0.0262 at $h = 5$, because each prediction is fed back and errors build up.

- Blank beats Copy last: frames are mostly black, and a digit copied to the wrong place creates error at both its copied location and its true location.
- **Overlap (Fig. 2, videos 851 and 877):** when digits cross, the model merges them into one grey blob. In video 851 it keeps the "3" for a few frames but loses the "1". After the crossing it cannot separate them again.

Q2.5 — Predictions before testing the baseline I committed a prediction template before testing the baseline (commit `d816e78`, Oct 28 13:05).

Reach estimate (Q2.3): 2 cells × 4 px = 8 px per coordinate per time step. Assumed typical training speed: approximately 3 px/frame → 2× ≈ 6 and 3× ≈ 9 px/frame.

- **Mirror test:** I expect model A to fail full consistency. Motion directions may already be roughly symmetric in training, but mirrored digits look unfamiliar, and the architecture does not enforce mirror symmetry.
- **Rotation test:** I expect failure, with greater degradation than mirroring, because rotating digits changes their appearance more substantially and the architecture does not enforce rotation symmetry.
- **Speed:** I expect deterioration at 2× and substantial breakdown at 3×. Under my assumed training speed, 2× corresponds to approximately 6 px/frame, whereas 3× corresponds to approximately 9 px/frame, placing greater pressure on the model's 8 px reach per coordinate. This is an approximate prediction rather than a strict threshold, since displacement direction, faster examples, and unfamiliar speeds also affect performance.

3. Testing the Baseline

Q2.6 — Reading C_T alone

- Blank and Copy last have $C_M = C_R = 0$ but are poor forecasters (E_M, $E_R = 0.0426$ and 0.0641).
- So a small C_T only means "consistent", not "correct". A model can be consistently wrong.
- E_T must be shown next to C_T to tell whether the consistent prediction is also accurate.

Q2.7 — One video from Fig. 3 (test video 168, $T = R$, $C_R = 0.00743$)

- **Truth (row 2):** the two rotated digits move apart; the upper digit drifts down and right, the lower one moves right.
- **Row 3, $f(R(X))$:** the model drags the upper digit downward too fast. By horizons 4–5 it has merged with the lower digit into one tall blob. One digit is effectively lost.
- **Row 4, $R(f(X))$:** the two digits stay apart and almost stop moving, then blur.
- So on the rotated clip the model predicts different motion (a fast downward slide and a merge) than it does on the original clip. Its motion rule depends on how the digits look.

Q2.8 — Is rotation harder than mirroring?

- **Rotation is harder for A:** $C_R = 2.95 \times 10^{-3}$ versus $C_M = 1.82 \times 10^{-3}$, about 1.6 times larger. The transformed error is also higher: $E_R = 0.0237$ versus $E_M = 0.0227$.
- Figure 1 suggests that both transformed motion distributions are reasonably familiar.

- Appearance is therefore a plausible explanation: digits such as 0, 1 and 8 can remain familiar under mirroring, while a 90° rotation changes most digits substantially.

Q2.9 — Faster motion (Table 4, Fig. 5)

- Model A degrades sharply at **2**×: $E^{(2)}$ rises from 0.0185 (1×) to 0.0457 (2×) and 0.0533 (3×). At 2× it is no better than predicting a blank frame (about 0.043).
- The median displacement alone does not explain this failure. 2× is about 6.6 px/frame, inside the 8 px reach. The results suggest unfamiliar motion is a major cause of the 2× failure.
- The comparison is only approximate: reach is per coordinate (8 px in x and in y, up to 11.3 px diagonally), while flow speed is a Euclidean length. 8 px is not a strict failure threshold.
- What the estimate missed: reach says what the model *can* connect, not what it has *learned*. It never saw these speeds. Also a digit is about 20 px wide, so even a 10 px jump stays within reach of part of the old digit.
- **Fig. 5 failure (test video 21, a "1" and a "4"):**
  - 1×: both digits recognisable and in the right place, slightly blurred.
  - 2×: the "4" *lags*: by $h = 2$ it is still close to its $h = 1$ spot while the true "4" has moved further up. The "1" smears downward. The model keeps moving digits at roughly its learned 1× step.
  - 3×: the "1" becomes a long vertical *smear* and the "4" fades into a faint blob, so both digits lose their shape. The model cannot tell where they went, so it spreads its guess over the possible positions (blur is what MSE favours when unsure).

Q2.10 — Unseen speed or limited reach?

- Model A alone cannot separate unfamiliar motion from limited reach. It was never trained on faster clips.
- Train C with the same architecture on all three speeds. Improvement would show that exposure to faster motion helps without changing reach.
- Compare C with D under the same training protocol. A further gain from D would support a role for reach, although architectural changes can also affect optimisation.
- A stronger follow up would test larger displacements, such as 4–5× playback, using longer source videos so that five inputs and at least two targets exist.

4. What Can Data Fix?

Q2.11 — Predictions before training B–D I committed a prediction template before testing the baseline (commit `d816e78`, Oct 28 13:05).

- **Model B:** Symmetry augmentation should reduce mirror and rotation failures, but will not guarantee exact consistency or directly fix fast motion failures.
- **Model C:** Speed augmentation should improve predictions at 2× and 3× when unfamiliar motion is the cause, but its unchanged reach may still limit fast motion tracking.
- **Model D:** Its larger reach should improve fast motion prediction beyond C if reach is the bottleneck, but mirror and rotation failures may remain because neither C nor D uses symmetry augmentation.

Q2.12 — Model D and its reach

- **Design:** model A with dilation 1 in layer 1 and **dilation 3** in layer 2. Everything else is unchanged.
- **Parameters:** 480,784 (dilation adds no weights), well under the limit of 721,176.
- **Reach:** input (or $h^{(1)}_{\tau-1}$) → layer 1 (1 cell) → layer 2 (3 cells) = 4 cells. The path from $h^{(2)}_{\tau-1}$ gives 3 cells. **Reach = 4 cells × 4 px = 16 px per step**, vs 8 px for model A.
- **No holes:** offsets $\{-1, 0, 1\}$ plus $\{-3, 0, 3\}$ cover every offset from -4 to 4. (Dilation 2 in both layers would also give 16 px but miss odd offsets.) The impulse test confirmed 4 cells with no unreachable cells.
- Chosen on reasoning about reach; it was not tuned on the test split.

Q2.13 — Labelling the failures (Table 5)

- **Mirror/rotation gap → appearance.** B (symmetry augmentation, same architecture) reduces the gaps: C_M 1.82e-3 → 1.08e-3, C_R 2.95e-3 → 1.44e-3, and E_R 0.0237 → 0.0227. It is still not zero.
- **2× failure → unfamiliar motion.** C (speed augmentation, same architecture, same reach) cuts $E^{(2)}$ at 2× from 0.0457 to 0.0290.
- **3× failure → mainly unfamiliar motion.** C reaches 0.0335 at 3× and D (double reach) reaches 0.0339: no gain from extra reach. This suggests reach was not the main limit at these speeds, but it does not rule reach out: D also changed how small motions are represented.
- The "C bad at 3×, D good" case **did not happen here**. At about 10 px/frame the motion is only just beyond 8 px, and a 20 px digit still overlaps its old position.
- **Why data cannot replace reach:** training can only change the weights on paths that exist. If the new position is further away than the network's reach, no weight setting connects old and new positions in one step, however much data you add.

Q2.14 — Cost on original 1× clips

- **B:** a small cost at 1×; $E^{(5)}$ rises from 0.0220 to 0.0227.
- **C:** error rises to 0.0266, but the faster clips improve substantially. This is a useful tradeoff if several playback speeds matter.
- **D:** error rises to 0.0300, and D does not beat C at any tested speed. The extra reach does not justify the accuracy loss in these experiments.
- Dilation changes how local motion is represented, and D was still improving late in training. These are possible explanations, not isolated causes.
- D's small consistency gaps should be read alongside its errors and blurred predictions. Low disagreement alone does not mean accurate motion.

5. The Final Model

Why $C_M = C_R = 0$ exactly. For any $h \in G$:

$$\bar{f}(hX) = \frac{1}{8} \sum_{g \in G} g^{-1} f(ghX).$$

Let $g' = gh$. As g runs over G, g' also runs over every element of G once (G is a group), and $g^{-1} = h\,g'^{-1}$. So

$$\bar{f}(hX) = \frac{1}{8}\sum_{g'\in G} h\,g'^{-1} f(g'X) = h\,\bar{f}(X),$$

because h only rearranges pixels (it is linear) and can be taken out of the average. Taking $h = M$ or $h = R$ gives $\bar{f}(TX) = T\bar{f}(X)$, so $C_T = 0$ for any f. Our measured values (about 2–4 $\times 10^{-17}$, Table 6) are floating point rounding only.

Choice of final model.

- Final model = **symmetrised D**, as the handout specifies.
- We also trained D+B. On the validation split it was close to D ($E^{(5)}$ 0.0298 vs 0.0297), so we kept D, which had slightly lower validation error.

Q2.15 — Learned vs built in symmetry

- **Consistency:** B has gaps near 10^{-3}; the wrapped models reach rounding error near 10^{-17}. Symmetrisation guarantees equivariance; augmentation encourages it.
- **Accuracy:** symmetrised B improves from 0.0227 to 0.0221, close to A's 0.0220. Symmetrised D changes little, from 0.0300 to 0.0299. Original and transformed errors are equal for each wrapped model.
- **Cost:** D takes 0.369 s for 1,000 videos; symmetrised D takes 2.94 s, about eight times longer. The wrapper requires no extra training.
- **Possible downside:** conflicting predictions can average into blur, especially if the task is not symmetric. Average error improves slightly here; individual clips may still worsen.

Q2.16 — Video 168 again (Fig. 6a)

- **Consistency is fixed:** rows 3 and 4 agree because of symmetrisation.
- **Motion is still wrong:** the predicted upper digit moves down and merges with the lower one by horizons 4–5; the true digits separate. Predictions are blurrier than A's, and Figure 6b also shows faint digits.
- This demonstrates the limits of the final symmetrised D on this clip. Figure 6a does not show C, so it cannot isolate the effects of speed training and increased reach.

Q2.17 — Final verdict

- **Verdict: partly.** Symmetrised D handles faster playback better than A: $E^{(2)}$ is 0.0308 at 2× and 0.0338 at 3×, versus 0.0457 and 0.0533. C's results show that training on varied speeds contributes strongly.
- **Evidence:** compare Tables 4–6 and Figure 6. Exact mirror and rotation consistency comes from the wrapper, so it does not by itself prove that the model learned motion.
- **Limits:** original speed error is worse (0.0299 versus 0.0220), predictions are blurred, and crossing digits still merge in video 168. Further tests should use speeds beyond training and check whether digits remain separate after crossing.

Appendix: Use of Generative AI

I used claude code and chatgpt to build LateX report and to verify if my results are good enough for submission. i also asked to check if i did everything correctly and nothing is wrong